Prompt to Payments

Stablecoin Edition · Solana

Tobiloba Adedeji · tobySolutions

Blockfest Africa · 22 Oct 2026 · Lagos





Tobiloba Adedeji

tobySolutions · @toby_solutions

Senior Technical Lead, Boh

Building EKGO!: driver, customer and ops apps for 500+ restaurants in California

Solana

Co-founded Solana Students Africa · Solana Foundation, Solana Developer Platform · Turbine Q4 2025 graduate

Before that

DevRel at Fleek and Gaia AI · Million.js (16K+ stars)

I ship with AI every day. I read every line it writes that touches money.

Build it. Break it. Fix it.

1

Build

Prompt an AI. Get a working app.

15 min

2

Break

Find what it got wrong.

20 min

3

Fix

Make the server check the chain.

25 min

4

x402

Let an agent pay your API per request.

20 min

About 70 of the 90 minutes are hands-on.

Get these before we prompt

1	**A wallet**	Phantom, Solflare or Backpack. Switch it to **devnet**.
2	**Devnet SOL**	`faucet.solana.com` · 2 requests per 8 hours
3	**Devnet USDC**	`faucet.circle.com` · Solana Devnet · 20 USDC, once per 2 hours
4	**Node 24 + an AI assistant**	Claude Code, Cursor, Copilot, any of them

> **Faucet down or rate limited?**
> Raise your hand. I have pre-funded devnet wallets and a workshop RPC URL.

Open your AI tool. Paste this.

You

Build a Next.js app where a customer connects a Solana wallet and pays 2.50 USDC on devnet for a jollof rice order. Show a success message when they've paid. The merchant wallet is <your wallet address>.

10:00

Get it running. Pay for one jollof. Raise your hand when you see the success message.

What came back

AI FIRST DRAFT

Files	6
Next.js	15.5
@solana/web3.js	1.99 (v1)
wallet-adapter	0.15 / 0.19
tsc --noEmit	passed
Time	about 4 min

IT BUILDS

Connect Phantom. Click pay. Approve.

"Payment successful! Your jollof rice is on the way."

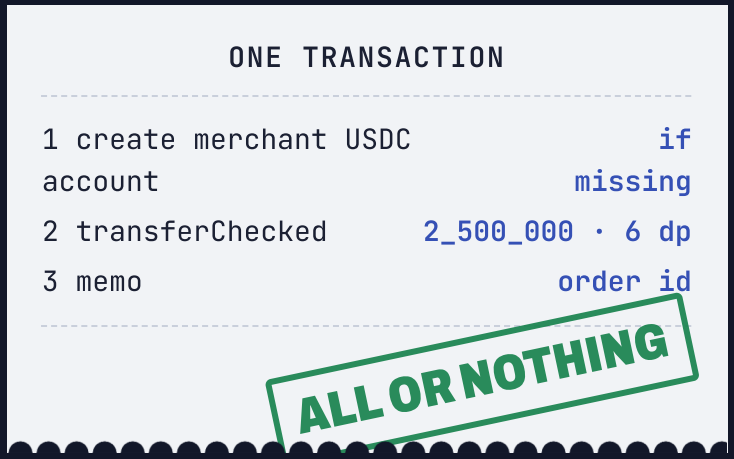
So we ship it?

2

Break it

It works. That’s a different question from whether it’s correct.

One payment, three instructions



Idempotent create · no-op if it already exists

transferChecked · the program checks the mint and decimals too

Memo · ties this payment to one order

Atomic · all three happen, or none do

LIVE → Editor · app/page.tsx

Read it like the money is yours

1. Who decides the order is paid?
2. Where does the 2.50 come from?
3. Two people pay 2.50 at once. Which jollof is whose?

5:00

The scorecard

Got right

- Correct devnet USDC mint, 6 decimals
- transferChecked, not transfer
- Creates the merchant account, idempotently
- Amount in base units: `2_500_000`

Got wrong

- The browser decides it's paid
- The price lives in the browser
- No order id: which jollof is this?
- Confirms with the wrong blockhash
- Assumes the classic token program
- A stack the Foundation now says not to start with

Changed since your AI was trained

Jul	**Solana Foundation's AI dev skill: don't start new apps on framework-kit or wallet-adapter**	Our AI first draft still used wallet-adapter
Sep	**Rent cut, first 2 of 5 steps**	A token account now costs 0.00148844 SOL, not the 0.00203928 in older code and docs
15 Sep	**v1 transactions on mainnet**	getTransaction without maxSupportedTransactionVersion: 1 errors on them
9 Oct	**Slots 400 ms → 200 ms**	A blockhash now expires in ~30 s, not ~60 s
21 Oct	**Alpenglow on mainnet**	Finality target ~150 ms. "confirmed" and "finalized" become the same thing

My AI got one wrong too

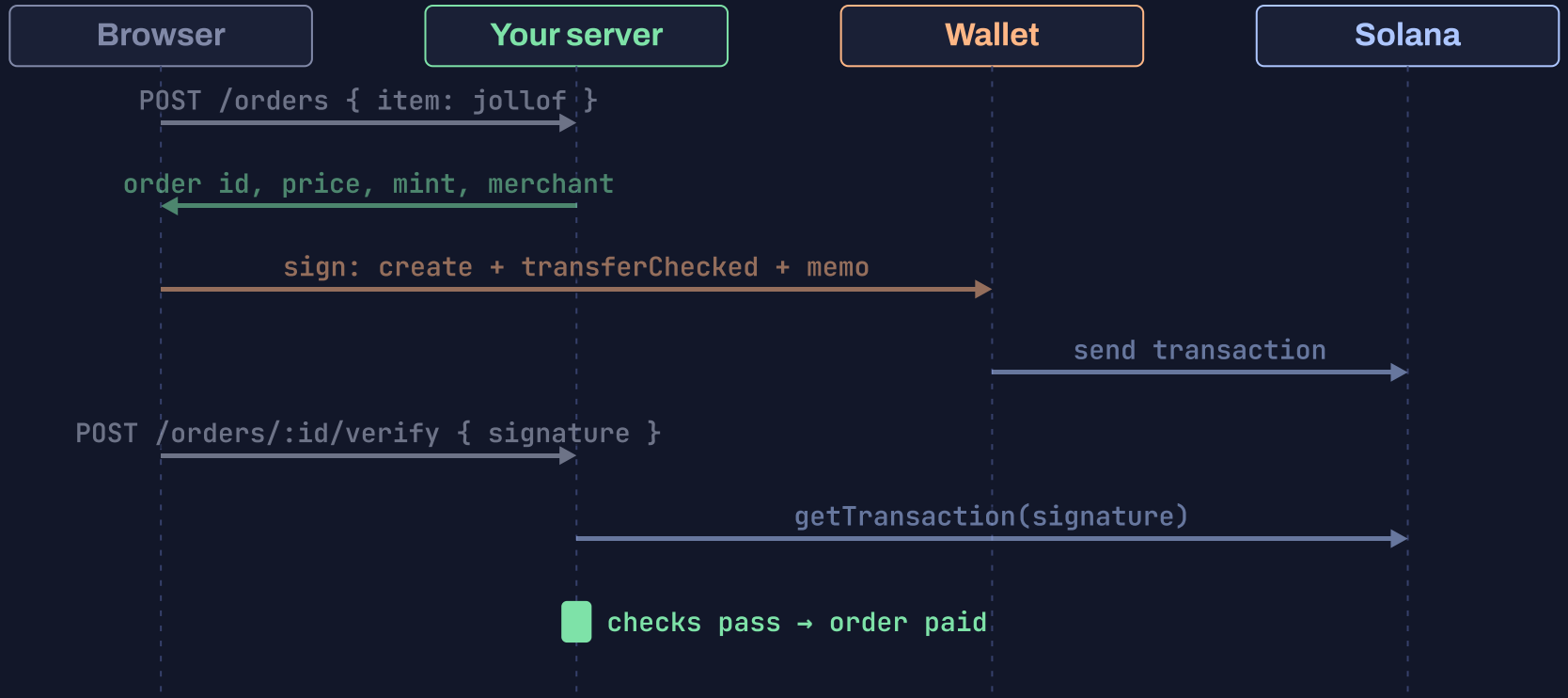
typed from memory
TokenzQdBNbLqP5VEhdkAS6EPFLC1PeFLLyVWSyU6Jkkt5sVhpWv

the real Token-2022 program
TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb

Never retype an address. Import it.

```
import { TOKEN_2022_PROGRAM_ADDRESS } from '@solana-program/token-2022'
```

The fix: the server checks the chain



Money is an integer

USDC has 6 decimals. 2.50 USDC is `2_500_000` base units. Try a price.

Price in USDC: 2.01

2.50 | 2.01 | 4.02 | 0.10

AI: `price * 1e6`

2009999.9999999998

not a whole number of base units

Parse the string as bigint

2010000n

exact, every time

LIVE → Editor

Fix it

1. Add an `/api/orders` route. The server sets the price.
2. Put the order id in a memo
3. Add `/verify`: fetch the tx, it succeeded, memo matches, merchant got paid
4. Refuse a signature you've already used

? Stuck? `app/` in the repo is the reviewed version

15:00

LIVE → Terminal · app/

Now attack it

```
npm run dev
node scripts/e2e.ts
```

Real devnet payments. Same routes the browser uses.

Ask your AI to write the attacks, then make them fail.



4

x402

When the customer is a program, not a person.

402 has been waiting since 1997

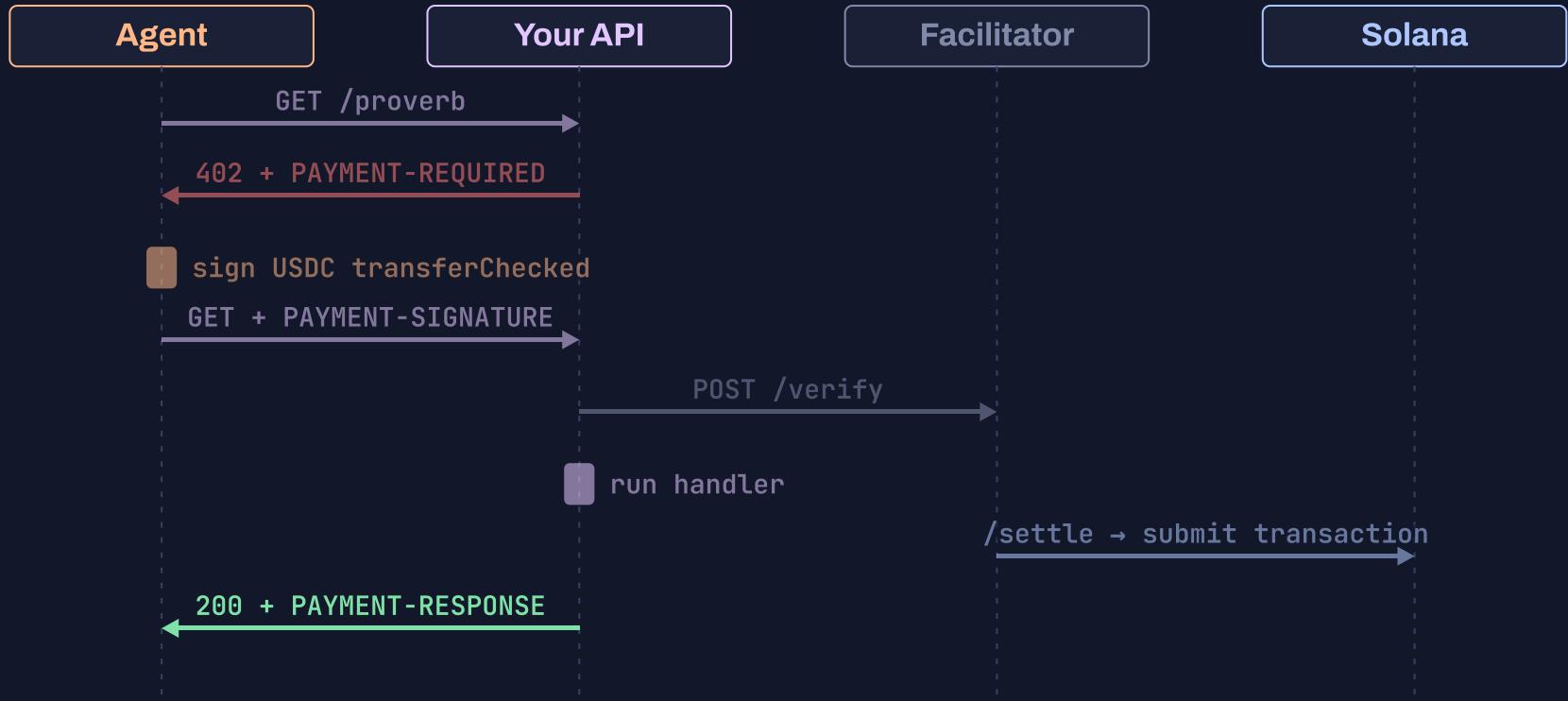
`HTTP/1.1 402 Payment Required`

"This code is reserved for future use." · RFC 2068, 1997

Still "reserved for future use" in RFC 9110, 2022

x402 is that future: a standard for paying inside HTTP.

One request, paid



LIVE → Terminal · x402/

Pay for a proverb

```
npm run server                  # paid API on :4021
curl -i localhost:4021/proverb  # 402 + PAYMENT-REQUIRED
npm run agent                   # signs $0.001 USDC, retries, gets the proverb
```

402
and the price, in a header

Signed
by the agent. No SOL needed

200
plus the proverb and a receipt

What the AI gets wrong with x402

v1 headers

`X-PAYMENT` is now `PAYMENT-SIGNATURE`

Network names

`"solana-devnet"` is v1. v2 uses `solana:EtWT…qa1`

Silent unprotected route

In `withX402`, a typo in a keyed route path means the handler runs **free**

Deprecated packages

`x402-next`, `x402-fetch`: v1, security fixes only, and npm gives no warning

Charging for errors

Next.js `paymentProxy` charges even for failed responses. `withX402` only settles below 400

No merchant USDC account

The payer can't create it in an x402 payment. Create it first

Before AI-written payment code ships

1. The server sets the price and creates the order
2. The server fetches the transaction itself
3. It succeeded: `meta.err` is null
4. Right mint, right recipient, full amount, from balances
5. Memo or reference ties it to one order
6. One signature, one order: unique in the database
7. Amounts are bigint base units, never floats
8. Token program and decimals read from the mint
9. Addresses come from packages or the chain, never memory
10. No secrets behind `NEXT_PUBLIC_`
11. An attack script that fails, in CI
12. x402: v2 headers, a spend limit, pinned SDK

Thank you

Slides, reference app, AI first draft, x402 service, checklist

github.com/tobySolutions/prompt-to-payments

@toby_solutions · tobySolutions.dev

